

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

CENTRO UNIVERSITARIO DE JALAPA

FACULTAD DE INGENIERÍA EN SISTEMAS

Proyecto Final

App de Guia Turistica

Estudiantes:

Ana Laura de María Oliva Cruz 0907-23-9832
Sinthia Celeste Orellana Galeano 0907-23-19496
Aura Lucia Sandoval Pivaral 0907-23-19786
Derek Leonel Mármol Salguero 0907-23-13365
Daniel Alexander Ortiz Cabrera 0907-23-14054
Diego Alexander Contreras Duarte 0907-23-8207

Sección: A

Semestre: Segundo

Ciclo: Octavo

Catedrático: Ing. Marco Tulio Valdez Albizurez

Fecha: 24/10/2026

Contenido

1. Introducción4
- 1.1 Propósito del documento4
- 1.2 Descripción general4
- 1.3 Ficha del sistema4
- 1.4 Funcionalidades principales5

2. Arquitectura del sistema6
- 2.1 Visión general6
- 2.2 Componentes6
- 2.3 Estructura del repositorio7
- 2.4 Flujo de generación de un itinerario8
- 2.5 Flujo durante el viaje8
- 2.6 Tipos compartidos9
- 2.7 Caché y consumo de APIs externas10
- 2.8 Autenticación y seguridad10
- 2.9 Manejo de errores10

3. Modelo de datos12
- 3.1 Modelos y campos12
- 3.2 Decisiones de diseño14
- 3.3 Migraciones14

4. Referencia de la API16
- 4.1 Convenciones16
- 4.2 Resumen de endpoints del backend16
- 4.3 Autenticación y usuarios17
- 4.4 Lugares18
- 4.5 Itinerarios19
- 4.6 Clima22
- 4.7 Hoteles24
- 4.8 Restaurantes24
- 4.9 Asistente conversacional25
- 4.10 Servicios externos utilizados25
- 4.11 API del optimizador26
- 4.12 Códigos de error del backend27

5. Instalación y despliegue28
- 5.1 Requisitos28
- 5.2 Claves de API28
- 5.3 Puesta en marcha paso a paso29
- 5.4 Variables de entorno30
- 5.5 Comandos útiles31
- 5.6 Probar en el teléfono31
- 5.7 Compilación de producción32

5.8 Solución de problemas32

6. Frontend33

6.1 Tecnologías33
6.2 Estructura de frontend/src33
6.3 Flujo de pantallas33
6.4 Estado global y persistencia34
6.5 Servicios35
6.6 Hooks35
6.7 Componentes36
6.8 Aplicación web progresiva (PWA)36
6.9 Pruebas del frontend37

7. Pruebas y calidad38

8. Limitaciones y recomendaciones39

8.1 Limitaciones conocidas del producto39
8.2 Observaciones técnicas y recomendaciones39
8.3 Trabajo futuro sugerido40

Anexo. Documentos relacionados del repositorio41

1. Introducción

1.1 Propósito del documento

Este documento describe el diseño, la implementación y la puesta en marcha de **Guía Turística Inteligente**. Está dirigido a quienes necesiten entender, instalar, mantener o ampliar el sistema: integrantes del equipo, docentes que lo evalúen y futuros colaboradores.

Reúne en un solo lugar lo que hasta ahora estaba repartido entre el README del repositorio, el README del backend, la documentación del optimizador y de Valhalla, y el documento de justificación de tecnologías. Los detalles muy extensos de cada módulo (por ejemplo, todas las reglas de validación de parámetros) se resumen aquí y se enlazan con su documento de origen.

1.2 Descripción general

Guía Turística es una aplicación web progresiva (PWA), pensada para usarse desde el teléfono, que arma un itinerario de viaje por Guatemala. El usuario indica el destino, las fechas, con quién viaja, su presupuesto, cómo se va a mover y qué le interesa. El sistema busca lugares reales cercanos, revisa sus horarios, el pronóstico del clima y los tiempos de traslado por calle, y propone un plan día por día.

Durante el viaje, la aplicación acompaña al usuario: muestra su posición en el mapa, lo guía con indicaciones paso a paso hasta la siguiente parada, avisa si se acerca lluvia y propone alternativas bajo techo, recomienda hoteles y restaurantes según el presupuesto, y permite recalcular la ruta cuando algo cambia.

1.3 Ficha del sistema

Aspecto	Detalle
Tipo de aplicación	Aplicación web progresiva (PWA) instalable en el teléfono
Alcance geográfico	Guatemala (la geocodificación solo acepta destinos dentro del país y el mapa de rutas es el de Guatemala)
Lenguajes	TypeScript (frontend y backend) y Python (optimizador de rutas)
Servicios propios	Frontend, backend (API), optimizador, PostgreSQL, Redis y Valhalla
Orquestación local	Docker Compose (PostgreSQL, Redis, optimizador y Valhalla); backend y frontend se ejecutan con npm
Control de versiones	Git y GitHub (repositorio DerekMarmol/GuiaTuristicaProyecto)

Aspecto	Detalle
Gestión de tareas	Tickets con identificador GUI-xxx, que aparecen citados en los comentarios del código
Idioma de la interfaz y de la API	Español (nombres de campos, mensajes y códigos de error)

1.4 Funcionalidades principales

Función	Qué hace	Dónde se implementa
Generación de itinerarios	Busca lugares reales, consulta horarios y arma los días con IA (Gemini) o, si no está disponible, con un planificador de respaldo.	backend/src/itinerarios, backend/src/ai
Optimización de rutas	Calcula el mejor orden de visita respetando ventanas de horario, con tiempos reales por calle.	optimizer/, Valhalla
Clima	Pronóstico por día y clima actual; clasifica los lugares como bajo techo o al aire libre.	backend/src/clima, backend/src/openweather
Reordenamiento reactivo por clima	Si aparece lluvia o tormenta durante el día, propone alternativas techadas y un nuevo recorrido que el usuario acepta o rechaza.	backend/src/clima/reactivo, frontend/src/hooks/useMonitoreoClima.ts
Hoteles y restaurantes	Hoteles cercanos con precio convertido a quetzales (tope del 40 % del presupuesto diario) y restaurantes por parada (tope del 30 %).	backend/src/hoteles, backend/src/restaurantes
Navegación guiada	Ruta por calles con indicaciones en español; detecta desvíos y llegada.	frontend/src/services/navegacion.ts, Mapbox Directions
Lugares abiertos ahora	Lugares cercanos cuyo horario publicado indica que están abiertos en este momento.	backend/src/lugares, frontend/src/components/AbiertosAhora.tsx
Asistente conversacional	Interpreta peticiones («quita el museo») y propone ajustes. El backend está implementado; el frontend aún usa respuestas de demostración.	backend/src/chat, frontend/src/components/ChatFlotante.tsx

Función	Qué hace	Dónde se implementa
Cuentas y planes guardados	Registro, inicio de sesión, guardado de planes, recuperación en otro dispositivo y exportación a JSON.	backend/src/routes/auth.routes.ts, itinerarios.ts
Apoyo en ruta	Recalculo de la hora estimada de llegada (ETA), telemetría de velocidad por GPS, panel de presupuesto y calificación de paradas.	frontend/src/components, backend/src/eta
Funcionamiento sin conexión	Instalación como app, caché de archivos y aviso de falta de conexión.	vite-plugin-pwa, OfflineBanner.tsx

2. Arquitectura del sistema

2.1 Visión general

El sistema se divide en servicios con una responsabilidad clara cada uno. El frontend habla con el backend por HTTP (solo el mapa y las indicaciones de navegación van directamente a Mapbox). El backend coordina las APIs externas, la base de datos, la caché y el optimizador. El optimizador es el único que habla con Valhalla.

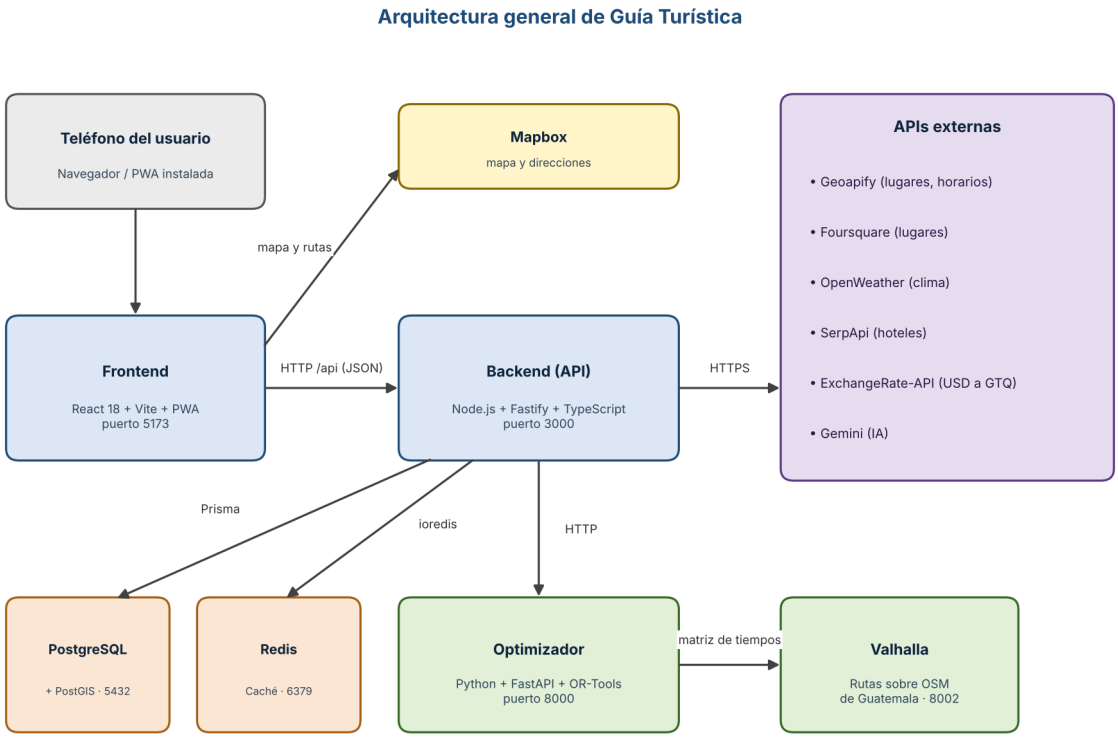


Figura 1. Arquitectura general y puertos de cada servicio

2.2 Componentes

Componente	Tecnología	Puerto	Responsabilidad
Frontend	React 18, Vite 5, TypeScript, Mapbox GL, vite-plugin-pwa	5173	Pantallas del flujo de 4 pasos, mapa, posición en vivo, navegación y funcionamiento como PWA.
Backend (API)	Node.js 20+, Fastify 5, TypeScript, Zod, Prisma 6	3000	Autenticación, generación y validación de itinerarios, consultas a APIs externas, IA y persistencia.
Optimizador	Python 3.12, FastAPI, Google OR-Tools	8000	Calcula el mejor orden de visita con ventanas de horario (problema de rutas con ventanas de tiempo).

Componente	Tecnología	Puerto	Responsabilidad
Valhalla	Imagen valhalla-scripted 3.9.1	8002	Motor de rutas propio con el mapa de Guatemala (OpenStreetMap): tiempos y distancias en auto o a pie.
PostgreSQL	Imagen postgis/postgis:16-3.4	5432	Usuarios, viajes, itinerarios, paradas y restricciones climáticas.
Redis	Imagen redis:7-alpine	6379	Caché de respuestas de APIs externas para ahorrar cuota y tiempo.

2.3 Estructura del repositorio

```
GuiaTuristicaProyecto/
├── backend/            API en Node.js + Fastify + Prisma
│   ├── prisma/         schema.prisma y migraciones
│   ├── src/
│   │   ├── routes/       rutas HTTP (auth, usuarios, itinerarios, lugares, clima, hoteles...)
│   │   ├── itinerarios/  generación del plan (candidatos, contexto, plan de respaldo)
│   │   ├── ai/           cliente Gemini, prompts, validadores de salida, esquemas Zod
│   │   ├── geoapify/     cliente de lugares, detalles, geocodificación y horarios
│   │   ├── foursquare/   segunda fuente de lugares
│   │   ├── openweather/  clima actual y pronóstico
│   │   ├── clima/        clasificación, priorización y clima reactivo
│   │   ├── hoteles/      SerpApi, tipo de cambio y regla del 40 %
│   │   ├── restaurantes/ búsqueda, tarifas y regla del 30 %
│   │   ├── chat/         agente conversacional con herramientas
│   │   ├── eta/          recálculo de hora estimada de llegada
│   │   ├── optimizador/  cliente HTTP del optimizador
│   │   ├── lib/redis.ts  caché
│   │   └── app.ts        ensamblado de la aplicación (inyección de dependencias)
│   └── test/           pruebas con node:test
├── frontend/           PWA en React + Vite
│   ├── src/
│   │   ├── pages/        Paso1Destino ... Paso4Generacion, PantallaAuth
│   │   ├── components/   mapa, paneles, chat, navegación, clima...
│   │   ├── hooks/        GPS, monitoreo del clima, navegación...
│   │   ├── services/     llamadas a la API y lógica de dominio
│   │   ├── context/      ViajeContext y AuthContext
│   │   └── utils/        fechas, horarios, almacenamiento local
│   └── test/           pruebas
├── optimizer/          Microservicio FastAPI + OR-Tools (app/, tests/)
├── shared/types/       Tipos TypeScript compartidos por frontend y backend
├── database/erd.md     Diagrama entidad-relación original (Mermaid)
├── docs/               Documentación (Valhalla, clima, PDF de tecnologías)
└── docker-compose.yml  PostgreSQL, Redis, optimizador y Valhalla
```

2.4 Flujo de generación de un itinerario

El usuario completa tres pasos de formulario y, al llegar al paso 4, el frontend llama a POST /api/itinerarios/generar. El backend ejecuta el siguiente proceso:

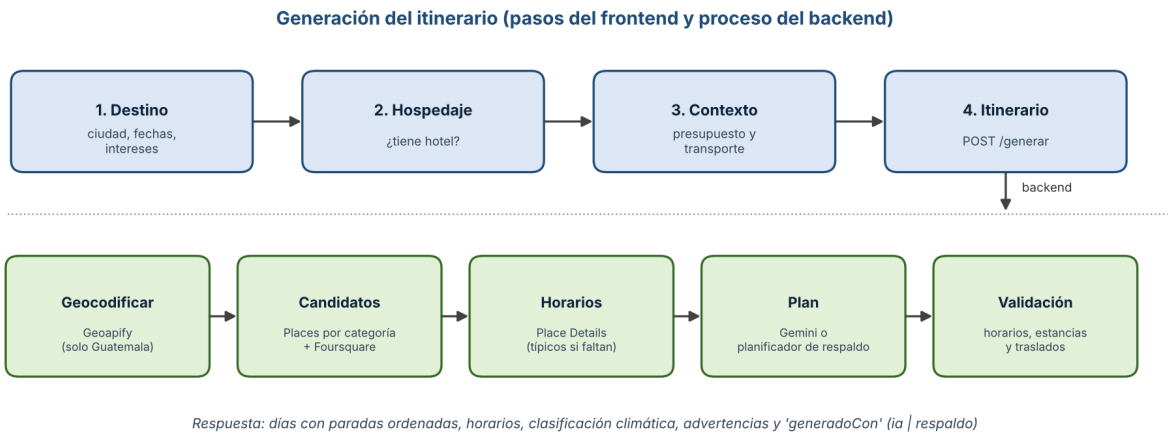


Figura 2. Pasos del frontend y proceso de generación en el backend

1. **Geocodificar el destino** con Geoapify, limitado a Guatemala. Si no hay un resultado con confianza de al menos 0.5, responde 422 DESTINO_NO_ENCONTRADO.
2. **Buscar candidatos:** una búsqueda de lugares por cada categoría de los intereses elegidos (radio de 15 km, o 3 km si el transporte es «A pie»), complementada con Foursquare cuando hay clave. Se descartan duplicados y se conservan entre 15 y 40 candidatos.
3. **Consultar horarios** de cada candidato con Place Details. En Guatemala muchos lugares no publican horario; a esos se les asigna uno típico de su tipo y la parada se marca con horarioEstimado: true.
4. **Armar el plan:** con GEMINI_API_KEY, lo arma Gemini y lo revisa un validador (generadoCon: "ia"). Sin clave, o si Gemini no logra un plan válido en 3 intentos, lo arma un planificador sin IA (generadoCon: "respaldo"). El pronóstico se usa como contexto para identificar los días con lluvia.
5. **Validar:** se comprueba que cada parada esté abierta a la hora estimada de llegada, que la estancia quepa en el horario y que los traslados sean coherentes. Lo que no cumple se informa en descartes y diasSinParadas.
6. **Responder** con los días, las paradas (con su clasificación tipo_entorno y sensibilidad_clima), las advertencias y el indicador viable.

El itinerario generado es un **borrador** (viajeId: "borrador"). Solo se guarda en la base de datos cuando el usuario con sesión iniciada lo guarda con POST /api/itinerarios/guardar.

2.5 Flujo durante el viaje

Reordenamiento reactivo por clima

Mientras el paso 4 está abierto, el día mostrado es hoy y quedan paradas pendientes, el frontend consulta al backend cada 12 minutos. El backend compara el riesgo actual con el de la revisión anterior y, si empeoró, busca alternativas techadas y calcula un nuevo recorrido. Nada se aplica sin que el usuario lo acepte.

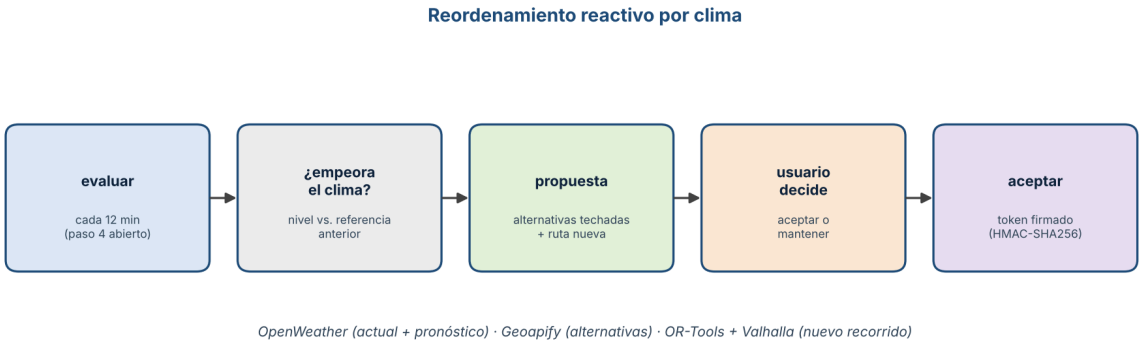


Figura 3. Ciclo del reordenamiento reactivo por clima

Mecanismo	Descripción
Propuesta firmada	La propuesta lleva un token firmado con HMAC-SHA256. Al aceptar, el backend verifica la firma, la vigencia (20 minutos) y que el itinerario no haya cambiado. El servidor no guarda propuestas.
Control de duplicados	Cada evento climático tiene un eventoId; el mismo evento no se propone dos veces y hay una espera mínima de 30 minutos entre propuestas.
Origen del recorrido	Se usa la ubicación que el usuario compartió (si tiene menos de 30 minutos), la parada en curso o la última completada; nunca el centro del destino.
Alternativas	Solo museos, lugares de cultura, planetarios y acuarios con horario confirmado. No se proponen lugares con horario desconocido.

Otros recálculos en ruta

- **Reoptimización** (POST /api/itinerarios/reoptimizar): recalcula el orden de las paradas pendientes ante un retraso, una parada cancelada o lluvia, llamando al optimizador.

- **ETA** (POST /api/itinerarios/recalcular-eta): recalcula la hora estimada de llegada a la siguiente parada y a las posteriores según el tiempo transcurrido.
- **Traslado inicial** (POST /api/itinerarios/traslado): estima el traslado desde el punto de partida del usuario hasta la primera parada.
- **Asistente** (POST /api/chat): interpreta peticiones en lenguaje natural; puede reoptimizar la ruta o proponer ajustes que el usuario debe confirmar.

2.6 Tipos compartidos

La carpeta shared/types contiene los contratos TypeScript que usan el frontend y el backend (Viaje, Parada, Itinerario, Restriccion, LugarInteres, PronosticoDiario, SolicitudRecomendacionesHoteles, climaReactivo, entre otros). Si cambia la forma de un dato, ambos lados lo detectan al compilar. El backend se compila con rootDir: ".." para poder importar esa carpeta, por lo que el servidor compilado queda en dist/backend/src/server.js.

2.7 Caché y consumo de APIs externas

Las APIs externas tienen cuotas gratuitas limitadas, por lo que el backend guarda sus respuestas en Redis cuando está disponible y limita la concurrencia de las consultas (por ejemplo, Geoapify admite 5 solicitudes por segundo en el plan gratuito).

Dato	Vigencia	Notas
Búsquedas de lugares (Geoapify)	24 horas	Clave de caché por consulta; ahorra créditos en búsquedas repetidas.
Lugares de Foursquare	24 horas	Segunda fuente, solo si hay FOURSQUARE_API_KEY.
Pronóstico de OpenWeather	30 minutos	Una consulta por viaje desde el paso 4.
Búsquedas de hoteles (SerpApi)	15 minutos	El precio es referencial.
Clima del monitoreo reactivo	10 minutos	Por zona de unos 110 m; es una caché por proceso, no una cuota global.
Aperturas de lugares	30 minutos	Para el modo «Abierto ahora».

Las pruebas automáticas reciben la caché como parámetro, así que no necesitan Redis ni consumen cuota.

2.8 Autenticación y seguridad

- **Contraseñas:** se guardan con hash bcrypt (10 rondas de sal); nunca en texto plano.
- **Sesión:** al iniciar sesión se emite un JWT con duración de 2 horas (JWT_EXPIRES_IN) y un refresh token de 7 días. Las rutas protegidas usan el decorador verificarToken y esperan el encabezado Authorization: Bearer <token>.
- **Propiedad de los planes:** un usuario solo puede ver, abrir o exportar los planes que le pertenecen; si pide el de otra persona recibe «no encontrado».
- **Claves de APIs:** viven solo en backend/.env y nunca se envían al frontend ni se definen con prefijo VITE_ (la excepción es el token público de Mapbox, que el navegador necesita).
- **Validación de entradas:** los parámetros se validan con Zod u objetos de esquema antes de consultar cualquier proveedor; los cuerpos tienen límites de tamaño (entre 4 KB y 100 KB según la ruta).

2.9 Manejo de errores

La mayoría de las rutas responden los errores con un formato común:

```
{
  "error": {
    "codigo": "PARAMETRO_INVALIDO",
    "mensaje": "Los parámetros de la consulta no son válidos.",
    "detalles": [ { "campo": "lat", "mensaje": "..." } ]
  }
}
```

Las rutas de autenticación, usuarios y guardado de planes usan otro formato, { "statusCode": 400, "mensaje": "..." }. Los errores no controlados responden 500 ERROR_INTERNO sin exponer detalles internos, y una ruta inexistente responde 404 RUTA_NO_ENCONTRADA.

3. Modelo de datos

La base de datos es relacional (PostgreSQL) y se gestiona con Prisma. El esquema está en backend/prisma/schema.prisma y tiene cinco modelos. Un usuario tiene muchos viajes; cada viaje se divide en días (itinerarios); cada día tiene paradas ordenadas; y cada parada puede tener una restricción climática.

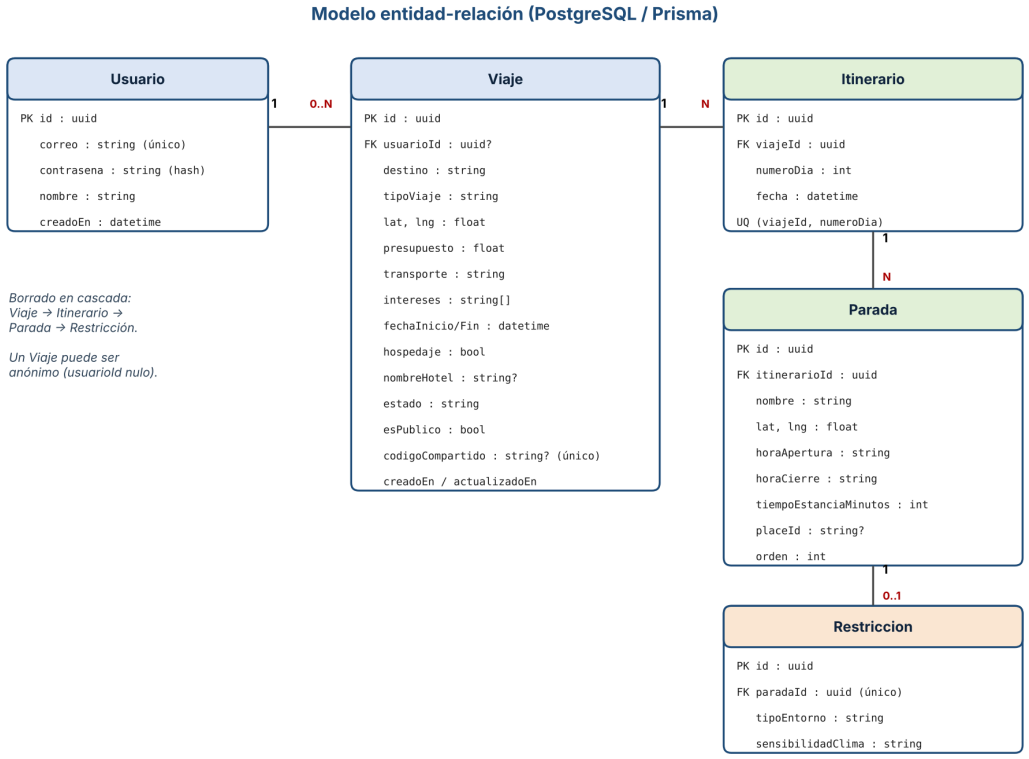


Figura 4. Diagrama entidad-relación

3.1 Modelos y campos

Usuario

Campo	Tipo	Notas
id	String (uuid)	Clave primaria
correo	String	Único; se guarda en minúsculas (la API lo normaliza)
contrasena	String	Hash bcrypt, nunca la contraseña en texto plano
nombre	String	
creadoEn	DateTime	Fecha de registro

Viaje

Campo	Tipo	Notas
id	String (uuid)	Clave primaria
usuarioId	String?	Clave foránea opcional: un viaje puede ser anónimo
destino	String	Texto del destino
tipoViaje	String	solo, pareja, familia o amigos
lat, lng	Float	Coordenadas del destino
presupuesto	Float	Presupuesto del viaje
transporte	String	A pie, Transporte Publico, Automovil, Moto o Transporte Aereo
intereses	String[]	Arreglo nativo de PostgreSQL
fechaInicio, fechaFin	DateTime	Rango del viaje
hospedaje, nombreHotel	Boolean, String?	Si ya tiene hospedaje y cuál
estado	String	en_curso (valor por defecto) o completado
esPublico, codigoCompartido	Boolean, String?	Preparados para compartir un plan con un código (único). Ver nota abajo
creadoEn, actualizadoEn	DateTime	Auditoría; índice por (usuarioId, creadoEn)

> **Nota.** Los campos esPublico y codigoCompartido existen en el esquema, pero en el código revisado ningún endpoint ni pantalla los utiliza todavía. La función de compartir un plan con un código está preparada en la base de datos, no implementada en la API.

Itinerario (un día del viaje)

Campo	Tipo	Notas
id	String (uuid)	Clave primaria
viajeId	String	Clave foránea; borrado en cascada
numeroDia	Int	Único junto con viajeId
fecha	DateTime	Fecha del día

Parada

Campo	Tipo	Notas
id	String (uuid)	Clave primaria
itinerarioId	String	Clave foránea; borrado en cascada
nombre	String	Nombre del lugar
lat, lng	Float	Coordenadas
horaApertura, horaCierre	String	Formato HH:MM
tiempoEstanciaMinutos	Int	Duración de la visita
placeId	String?	Identificador del lugar en el proveedor (Geoapify)
orden	Int	Posición dentro del día (0, 1, 2…); valor por defecto 0

Restriccion

Campo	Tipo	Notas
id	String (uuid)	Clave primaria
paradaId	String	Clave foránea única (una restricción por parada); borrado en cascada
tipoEntorno	String	Bajo Techo o Al aire libre
sensibilidadClima	String	baja, media o alta

3.2 Decisiones de diseño

- **Borrado en cascada.** Al borrar un viaje se eliminan sus días, paradas y restricciones; lo garantiza la propia base de datos.
- **Valores enumerados como texto.** Los valores permitidos (tipoViaje, transporte, estado…) se guardan como String y se validan en la API, no con enumeraciones de la base de datos.
- **Coordenadas como latitud y longitud.** La imagen de PostgreSQL incluye PostGIS, de modo que las consultas espaciales podrían hacerse en la base sin cambiar de motor; hoy las consultas de cercanía se resuelven en las APIs externas.

- **Planes anónimos y borrado de usuarios.** usuarioId es opcional y, si se borra el usuario, sus viajes se conservan con usuarioId nulo (ON DELETE SET NULL). Los endpoints de lectura y exportación solo devuelven planes de su dueño.

3.3 Migraciones

Las migraciones están en backend/prisma/migrations. Se aplican en orden con npx prisma migrate dev. Una migración ya subida a main no debe editarse: Prisma detecta el cambio de checksum y pide reiniciar la base.

Migración	Cambio
20260929044854_init_modelos_compartidos	Modelos iniciales: Viaje, Itinerario, Parada y Restriccion.
20261013045400_agregar_usuario	Tabla Usuario.
20261013154546_viaje_usuario_historial	Agrega a Viaje creadoEn, estado y usuarioId (clave foránea con ON DELETE SET NULL) e índice por (usuarioId, creadoEn).
20261013221940_disenar_esquema_itinerarios_historial	Agrega actualizadoEn, esPublico y codigoCompartido (único); hace único (viajeId, numeroDia) y recrea las claves foráneas con borrado en cascada.
20261014000000_agregar_place_id_parada	Campo placeId en Parada.
20261022000000_agregar_orden_parada	Campo orden en Parada.

4. Referencia de la API

4.1 Convenciones

Aspecto	Detalle
URL base (desarrollo)	http://localhost:3000. El frontend usa rutas relativas /api/... y Vite las reenvía al backend.
Formato	JSON en español. Las fechas usan AAAA-MM-DD y las horas HH:MM.
Autenticación	Rutas marcadas «Sí»: encabezado Authorization: Bearer <token>. Sin token válido responde 401.
Coordenadas	lat entre -90 y 90; lng entre -180 y 180, en notación decimal simple (sin exponentes, NaN ni valores vacíos).
Errores	Formato { error: { codigo, mensaje, detalles? } } (ver sección 4.12).
Límite de cuerpo	Entre 4 KB y 100 KB según la ruta. Un cuerpo mayor o JSON inválido responde 400 PARAMETRO_INVALIDO.
Contratos TypeScript	shared/types/: hotel.ts, restaurante.ts, climaReactivo.ts, clima.ts, lugarInteres.ts…

4.2 Resumen de endpoints del backend

El backend expone 26 endpoints. El optimizador agrega 4 más (sección 4.11).

Método	Ruta	Auth	Descripción
POST	/api/auth/registro	No	Crea una cuenta
POST	/api/auth/login	No	Inicia sesión; devuelve token y refresh token
POST	/api/auth/renovar	No	Obtiene un token nuevo con el refresh token
GET	/api/auth/verificar	Sí	Comprueba que el token sea válido
GET	/api/usuarios/perfil	Sí	Datos del usuario autenticado
PUT	/api/usuarios/perfil	Sí	Actualiza nombre o correo
GET	/api/lugares	No	Lugares cercanos por categoría o interés, con horario y clasificación climática

Método	Ruta	Auth	Descripción
GET	/api/lugares/abiertos-ahora	No	Lugares cercanos abiertos en este momento
POST	/api/itinerarios/generar	No	Genera el itinerario completo
POST	/api/itinerarios/validar	No	Valida una propuesta contra los horarios reales
POST	/api/itinerarios/priorizar-clima	No	Prioriza lugares bajo techo en días con lluvia
POST	/api/itinerarios/guardar	Sí	Guarda el plan del usuario
GET	/api/itinerarios/mis-planes	Sí	Lista los planes guardados (hasta 20)
GET	/api/itinerarios/:id	Sí	Plan completo con paradas en orden
GET	/api/itinerarios/:id/exportar	Sí	Descarga el plan como archivo JSON
POST	/api/itinerarios/reoptimizar	No	Recalcula el orden de las paradas pendientes
POST	/api/itinerarios/recalcular-eta	No	Recalcula la hora estimada de llegada
POST	/api/itinerarios/traslado	No	Estima el traslado del punto de partida a la primera parada
POST	/api/itinerarios/clima-reactivo/evaluar	No	Evalúa el riesgo climático del día
POST	/api/itinerarios/clima-reactivo/propuesta	No	Propone alternativas techadas y nuevo recorrido
POST	/api/itinerarios/clima-reactivo/aceptar	No	Aplica una propuesta firmada
GET	/api/clima/pronostico	No	Resumen diario del clima para las fechas del viaje
GET	/api/clima/actual	No	Clima observado ahora en unas coordenadas
POST	/api/hoteles/recomendaciones	No	Hoteles cercanos dentro del 40 % del presupuesto diario
POST	/api/restaurantes/recomendaciones	No	Restaurantes por parada dentro del 30 % del presupuesto diario

Método	Ruta	Auth	Descripción
POST	/api/chat	No	Asistente conversacional con herramientas

4.3 Autenticación y usuarios

POST /api/auth/registro

Crea una cuenta. El correo se guarda en minúsculas y la contraseña como hash bcrypt.

```
// Solicitud
{ "nombre": "Ana López", "correo": "ana@correo.com", "contrasena": "********" }

// Respuesta 201
{ "mensaje": "Usuario registrado exitosamente",
  "usuario": { "id": "…", "nombre": "Ana López", "correo": "ana@correo.com", "creadoEn": "…" } }
```

Estado	Cuándo
400	Falta nombre, correo o contraseña, o el correo ya está registrado.

POST /api/auth/login

```
// Solicitud
{ "correo": "ana@correo.com", "contrasena": "********" }

// Respuesta 200
{ "mensaje": "Inicio de sesión exitoso",
  "token": "<JWT, 2 h>", "refreshToken": "<JWT, 7 días>",
  "usuario": { "id": "…", "nombre": "Ana López", "correo": "ana@correo.com" } }
```

Estado	Cuándo
400	Falta el correo o la contraseña.
401	Credenciales inválidas (el mensaje no distingue entre correo inexistente y contraseña errónea).

POST /api/auth/renovar

Recibe { "refreshToken": "…" } y devuelve { mensaje, token } con un token nuevo. Responde 401 si el refresh token falta, es inválido o expiró, o si el usuario ya no existe.

GET /api/auth/verificar

Ruta protegida de comprobación. Devuelve { mensaje: "Token válido", usuarioAutenticado }.

GET y PUT /api/usuarios/perfil

GET devuelve id, nombre, correo y creadoEn del usuario autenticado (nunca la contraseña). PUT acepta nombre y/o correo; responde 400 si no se envía ninguno o si el correo ya lo usa otra cuenta, y 404 si el usuario no existe.

4.4 Lugares

GET /api/lugares

Lugares cercanos a unas coordenadas, obtenidos de Geoapify, con su horario y su clasificación climática. Requiere al menos categoria o intereses.

Parámetro	Obligatorio	Regla
lat, lng	Sí	Coordenadas decimales válidas.
categoria	Uno de los dos	Una categoría admitida: entertainment.museum, entertainment.culture, catering.restaurant, catering.cafe, accommodation.hotel, tourism.attraction, tourism.sights, leisure.park.
intereses	Uno de los dos	Lista separada por comas de los diez intereses: Cultura, Historia, Gastronomía, Naturaleza, Aventura, Compras, Vida nocturna, Playa, Fotografía, Arte (sin distinguir mayúsculas ni tildes).
radio	No	Entero de 1 a 50000 metros. Por defecto 5000.
limite	No	Entero de 1 a 100. Por defecto 20.

Si se envían categoria e intereses, se consulta la unión de ambos en una sola petición a Geoapify. La respuesta es { lugares: [...] } y cada lugar incluye:


```
{ "idExterno": "51f0a6...", "proveedor": "geoapify",
  "nombre": "Museo Nacional de Arqueología y Etnología",
  "direccion": "Zona 13, Ciudad de Guatemala, Guatemala",
  "lat": 14.5907, "lng": -90.5242,
  "categorias": ["entertainment", "entertainment.museum"],
  "tipo_entorno": "indoor", "sensibilidad_clima": "baja", "regla_clima": "entertainment.museum",
  "horario": { "estado": "disponible", "valorOriginal": "Tu-Fr 09:00-16:00; Mo off", "fuente": "geoapify" } }
```


GET /api/lugares/abiertos-ahora

Devuelve los lugares cercanos cuyo horario publicado indica que están abiertos en el momento de la consulta, evaluados en la zona horaria del lugar y ordenados por cercanía. Los lugares cerrados o sin horario verificable se excluyen. «Abierto» es una evaluación del horario publicado, no una confirmación en tiempo real.

Parámetro	Obligatorio	Regla
lat, lng	Sí	Coordenadas decimales válidas.

Parámetro	Obligatorio	Regla
radio	No	1 a 10000 metros. Por defecto 2000.
limite	No	1 a 20. Por defecto 10.
categoria, intereses	No	Mismas reglas que /api/lugares. Sin ninguno, se usa un conjunto acotado de categorías turísticas (atracciones, museos, cultura y parques).

4.5 Itinerarios

POST /api/itinerarios/generar

Genera el itinerario completo (ver sección 2.4).

```
// Solicitud
{ "destino": "Puerto Barrios, Izabal", "fechaInicio": "2026-11-10", "fechaFin": "2026-11-12",
  "intereses": ["Playa", "Naturaleza", "Gastronomía"], "tipoViaje": "pareja",
  "transporte": "Automovil", "presupuesto": 2400, "presupuestoDiario": 800,
  "hospedaje": false, "horaInicio": "09:00" }
```

Son obligatorios destino, fechaInicio, fechaFin (máximo 14 días) e intereses. Los demás tienen valores por defecto (tipoViaje: "solo", transporte: "Automovil", presupuesto: 0, hospedaje: false, horaInicio: "09:00").

```
// Respuesta 200 (resumida)
{ "destino": { "nombre": "Puerto Barrios, IZ, Guatemala", "lat": 15.7275, "lng": -88.5952, "zonaHoraria":
"America/Guatemala" },
  "horaInicio": "09:00", "generadoCon": "respaldo", "viable": true,
  "itinerarios": [ { "id": "it-1", "viajeId": "borrador", "numeroDia": 1, "fecha": "2026-11-10",
      "paradas": [ { "id": "p1-1", "placeId": "51f0…", "nombre": "Playa Publico", "lat": 15.6944, "lng": -88.6243,
          "horaApertura": "06:00", "horaCierre": "18:00", "tiempoEstanciaMinutos": 120,
          "llegada": "2026-11-10T09:00-06:00", "salida": "2026-11-10T11:00-06:00",
          "tipo_entorno": "outdoor", "sensibilidad_clima": "alta", "horarioEstimado": true } ] } ],
  "descartes": [], "diasSinParadas": [], "advertencias": [ "1 parada(s) no publican horario…" ] }
```

Estado	Código	Cuándo
400	PARAMETRO_INVALIDO	Fechas, intereses o duración inválidos (más de 14 días).
422	DESTINO_NO_ENCONTRADO	El destino no se encontró en Guatemala.
422	SIN_LUGARES	No hay lugares cercanos para esos intereses, o ninguno abre en las fechas.
5xx	PROVEEDOR_*, SERVICIO_NO_CONFIGURADO	Falla o falta de clave de un proveedor externo.

POST /api/itinerarios/validar

Recibe una propuesta (viaje con id, fechas, zonaHoraria IANA y horaInicio; y propuesta.dias[].paradas[] con placeId y tiempoEstanciaMinutos de 15 a 360), consulta los horarios reales en Geoapify y devuelve el plan sin las paradas que estarían cerradas a la hora estimada de llegada. Admite de 1 a 5 paradas por día y hasta 20 lugares distintos por solicitud. El frontend actual no lo consume.

POST /api/itinerarios/priorizar-clima

Misma propuesta que /validar, más las coordenadas del destino. Clasifica cada lugar como indoor u outdoor, coloca primero los lugares bajo techo en los días con lluvia pronosticada y después valida los horarios sobre ese orden. Si falta OPENWEATHER_API_KEY, responde 200 con el orden original e informa la consulta fallida. El frontend actual no lo consume.

POST /api/itinerarios/guardar

Requiere sesión. Guarda de forma anidada el viaje, sus días, paradas y restricciones asociadas al usuario autenticado. Acepta el plan dentro de plan o directamente en el cuerpo.

```
// Solicitud (campos principales)
{ "destino": "Antigua Guatemala", "tipoViaje": "pareja", "presupuesto": 1500, "transporte": "A pie",
  "intereses": ["Cultura"], "fechaInicio": "2026-11-20", "fechaFin": "2026-11-21", "lat": 14.5586, "lng": -90.7295,
  "hospedaje": false,
  "dias": [ { "numeroDia": 1, "fecha": "2026-11-20",
      "paradas": [ { "nombre": "Catedral", "lat": 14.5586, "lng": -90.7295, "horaApertura": "08:00",
          "horaCierre": "18:00", "tiempoEstanciaMinutos": 45, "placeId": "…",
          "restriccion": { "tipoEntorno": "Bajo Techo", "sensibilidadClima": "baja" } } ] } ] }

// Respuesta 201
{ "mensaje": "Plan de viaje e itinerario guardados exitosamente", "id": "…", "viajeId": "…", "viaje": { … } }
```

Responde 400 si falta el destino o no hay al menos un día, y 401 sin token.

GET /api/itinerarios/mis-planes

Requiere sesión. Devuelve hasta 20 planes del usuario, del más reciente al más antiguo: { planes: [{ id, destino, fechaInicio, fechaFin, creadoEn, totalDias, totalParadas }] }.

GET /api/itinerarios/:id

Requiere sesión. Devuelve { plan } con los días en orden y las paradas ordenadas por orden, cada una con su restriccion. Responde 404 si el plan no existe o pertenece a otro usuario.

GET /api/itinerarios/:id/exportar

Requiere sesión. Devuelve un archivo JSON descargable (Content-Disposition: attachment, nombre itinerario_<destino>_<id>.json) con metadatos, los datos del viaje, notas generales y, por día, las paradas con coordenadas, horarios, estancia y restricción climática.

POST /api/itinerarios/reoptimizar

Recalcula el orden de las paradas pendientes llamando a POST {OPTIMIZER_URL}/reoptimize. Elimina la parada cancelada (si aplica) y convierte los campos al contrato del optimizador.

Campo	Descripción
tipo	clima, retraso o parada_cancelada.
paradas	Paradas pendientes del día.
horaActualMinutos	Minutos desde la medianoche.
modoTransporte	Opcional: auto (por defecto) o pedestrian.
origen	Opcional: { lat, lng } desde donde se reanuda el recorrido.
paradaCanceladaId	Solo para parada_cancelada.

Estado	Código	Cuándo
400	PARAMETRO_INVALIDO	Datos inválidos; no se llama al optimizador.
409	REOPTIMIZACION_NO_DISPONIBLE	No hay un orden viable con los horarios y rutas disponibles.
502	OPTIMIZADOR_NO_DISPONIBLE, RESPUESTA_OPTIMIZADOR_INVALIDA	No se pudo conectar con el optimizador o devolvió un orden inválido.
503	SERVICIO_RUTAS_NO_DISPONIBLE	Valhalla no está disponible.

POST /api/itinerarios/recalcular-eta

Recalcula la hora estimada de llegada. Campos obligatorios: horaActual (texto), tiempoTranscurridoMinutos, tiempoTrasladoPrevistoMinutos y tramos[] con paradaId, tiempoTrasladoMinutos y tiempoEstanciaMinutos. Devuelve el resultado del recálculo (incluida etaActualizada) y proximasParadas con la nueva ETA de cada parada siguiente. Un dato inválido responde 400.

POST /api/itinerarios/traslado

Recibe { origen: { lat, lng }, destino: { lat, lng } } (cuerpo de hasta 4 KB) y devuelve { traslado } con la estimación desde el punto de partida hasta la primera parada, calculada con el mismo estimador que los tramos entre paradas del itinerario.

4.6 Clima

GET /api/clima/pronostico

Resumen diario de temperatura y lluvia para cada fecha del viaje, a partir del pronóstico de 5 días / 3 horas de OpenWeather. Hace una sola petición al proveedor (10 s de espera máxima) y aplica después el rango del viaje.

Parámetro	Obligatorio	Regla
lat, lng	Sí	Coordenadas decimales válidas.
fechaInicio, fechaFin	Sí	Fechas reales AAAA-MM-DD; fechaFin igual o posterior. Máximo 366 días de rango.

Las fechas fuera del horizonte del pronóstico (unos 5 días) o pasadas aparecen como fuera_de_cobertura; no se consultan datos históricos. Sin OPENWEATHER_API_KEY responde 503.

GET /api/clima/actual

Temperatura y condición observadas en este momento. Parámetros: lat y lng. Respuesta:

```
{ "ubicacion": { "lat": 14.5586, "lng": -90.7295 }, "observadoEn": "2026-11-20T16:02:00.000Z",
  "temperaturaC": 22.4, "descripcion": "nubes dispersas", "lluvia1hMm": 0 }
```

POST /api/itinerarios/clima-reactivo/evaluar y /propuesta

Aceptan JSON de hasta 64 KB con el itinerario del día y el modo de transporte. evaluar solo consulta el clima y no consume créditos de Geoapify; propuesta además busca alternativas y calcula el nuevo recorrido.

Campo de la solicitud	Regla
itinerario.id, itinerario.fecha	Identificador y fecha del día.
itinerario.paradas[]	1 a 25 paradas con id único, coordenadas, horas HH:MM y estado: completada, en_curso (máximo una), pendiente o cancelada.
modoTransporte	auto (por defecto) o pedestrian.
intereses	Opcional; mismos valores de /api/lugares.
ubicacionActual	Opcional; solo si el usuario la compartió. Se ignora si tiene más de 30 minutos.
referencia	La devuelta por la revisión anterior, sin modificar; null en la primera.

La respuesta de evaluar incluye estado, evaluacion (fuente, nivel de riesgo, señal principal, umbrales), requierePropuesta, motivo, mensaje, la nueva referencia, proximaEvaluacionSegundos y advertencias. propuesta agrega el objeto propuesta:

Estado de la propuesta	Significado	¿Se puede aceptar?
pendiente	Hay al menos una sustitución y una ruta viable.	Sí, con token
sin_cambios	Ninguna parada pendiente coincide con el riesgo.	No
sin_alternativas	No hay alternativas techadas con horario confirmado, o ninguna es viable.	No
sin_solucion_viable	Con las alternativas no existe un recorrido que respete los horarios.	No

Niveles de riesgo, de menor a mayor: favorable, lluvia_ligera, riesgo_elevado, lluvia_significativa y tormenta. Solo se propone un cambio cuando el nivel actual es mayor que el de la revisión anterior y es al menos riesgo_elevado (probabilidad de precipitación ≥ 70 % por defecto).

POST /api/itinerarios/clima-reactivo/aceptar

Recibe { token, itinerario } y devuelve paradasPendientes en el orden propuesto.

Situación	Respuesta
Mismo itinerario y no venció (20 minutos)	200, yaAplicada: false
La propuesta ya estaba aplicada	200, yaAplicada: true (idempotente)
El itinerario cambió (parada completada, cancelada o reordenada)	409 PROPUESTA_OBSOLETA
La propuesta venció	410 PROPUESTA_VENCIDA
Token alterado, de otra instancia o de un reinicio sin CLIMA_REACTIVO_SECRETO	400 PROPUESTA_INVALIDA

4.7 Hoteles

POST /api/hoteles/recomendaciones

Hoteles cercanos al centro de las paradas de un día, obtenidos de SerpApi (Google Hotels) para una noche, con el precio referencial en USD convertido a quetzales con ExchangeRate-API. Solo se recomiendan los que cuestan como máximo el **40 %** del

presupuesto diario. Requiere SERPAPI_API_KEY y EXCHANGERATE_API_KEY (503 si faltan). El precio es referencial y puede diferir del cargo final.

Campo	Oblig.	Regla
fecha	Sí	AAAA-MM-DD, fecha de entrada de la noche; no puede haber pasado.
destino	Sí	Texto de 2 a 100 caracteres con la ciudad o zona de búsqueda.
paradas	Sí	0 a 25 objetos { lat, lng } de ese día.
presupuestoDiario	Sí	En quetzales; mayor que 0 y hasta 1 000 000, con máximo dos decimales.
moneda	Sí	Solo GTQ.
ocupacion	Sí	Entero de 1 a 10: adultos en una habitación.
hospedaje	Sí	true si ya tiene hospedaje reservado.
sinHotel	No	true si no necesita hotel.
radio, limite	No	Radio de 1 a 50000 m (por defecto 5000); máximo de 1 a 20 resultados (por defecto 10).

Si el usuario ya tiene hospedaje o no necesita hotel, no se consulta ningún proveedor. Cuando no hay recomendaciones, la respuesta indica el motivo: sin_hoteles_con_precio, sin_hoteles_en_radio o sin_hoteles_en_presupuesto. El cuerpo no puede superar 32 KB y no se acepta un porcentaje enviado por el cliente.

4.8 Restaurantes

POST /api/restaurantes/recomendaciones

Restaurantes cercanos a cada parada del día (Geoapify), filtrados con tarifas referenciales en quetzales contra el presupuesto de alimentación, que es el **30 %** del presupuesto diario. Se evalúa una comida principal por día, con el precio máximo del rango (criterio conservador): el costo del grupo es el máximo por persona multiplicado por los comensales.

Campo	Oblig.	Regla
fecha	No	Solo se devuelve en la respuesta.
paradas	Sí	0 a 25 objetos { id, lat, lng }; id único de 1 a 100 caracteres.
presupuestoDiario	Sí	En quetzales; mayor que 0 y hasta 1 000 000.

Campo	Oblig.	Regla
moneda	Sí	Solo GTQ.
comensales	Sí	Entero de 1 a 10.
radio	No	1 a 10000 m alrededor de cada parada. Por defecto 2000.
limite	No	1 a 20 por parada. Por defecto 10.

4.9 Asistente conversacional

POST /api/chat

Interpreta un mensaje del viajero y responde con texto, ajustes propuestos y acciones ejecutadas. Usa Gemini con dos herramientas: reoptimizar_ruta (recalcula de verdad el orden de las paradas pendientes mediante el optimizador) y proponer_ajuste (solo propone un cambio que el usuario debe confirmar). Requiere GEMINI_API_KEY.

Campo	Regla
mensaje	Texto de 1 a 500 caracteres (obligatorio).
paradas	Hasta 20 paradas con id, nombre y, opcionalmente, coordenadas, horario y estancia.
historial	Hasta 10 mensajes previos con autor (usuario o asistente) y texto.
horaActual, presupuestoRestante, modoTransporte, origen	Opcionales.

```
// Respuesta 200
{ "respuesta": "Listo, reordené tu ruta…",
  "ajustes": [ { "tipo": "quitar_parada", "paradaId": "cerro", "motivo": "…" } ],
  "acciones": [ { "tipo": "reoptimizar_ruta", "estado": "aplicada", "ordenOptimo": ["museo","cerro"], "distanciaTotalKm": 3.4 } ]
}
```

Los tipos de ajuste son quitar_parada, cambiar_tiempo, reordenar y buscar_alternativa (máximo 5 por respuesta). Una acción puede quedar en estado no_alcanza si ya no se llega a tiempo a todo.

4.10 Servicios externos utilizados

Servicio	Uso	Variable de entorno
Geoapify	Lugares, detalles y horarios, geocodificación	GEOAPIFY_API_KEY (obligatoria)
Foursquare	Segunda fuente de lugares (comercios y restaurantes)	FOURSQUARE_API_KEY
OpenWeather	Pronóstico y clima actual	OPENWEATHER_API_KEY
SerpApi	Hoteles (Google Hotels)	SERPAPI_API_KEY
ExchangeRate-API	Conversión de USD a quetzales	EXCHANGERATE_API_KEY
Google Gemini	Generación de itinerarios y asistente	GEMINI_API_KEY, GEMINI_MODEL
Mapbox	Mapa y direcciones (desde el navegador)	VITE_MAPBOX_TOKEN, VITE_MAPBOX_STYLE_URL

4.11 API del optimizador

Servicio FastAPI en el puerto 8000. Antes de optimizar, consulta a Valhalla una sola matriz de tiempos y distancias por calle entre todas las paradas; no usa distancias en línea recta. El detalle completo está en optimizer/API.md y docs/valhalla.md.

Método	Ruta	Descripción
GET	/health	Comprueba que el servicio esté activo; responde { "status": "ok" }.
POST	/optimize	Recibe paradas con horarios y estancia; devuelve orden_optimo y distancia_total_km.
POST	/reoptimize	Igual que /optimize para las paradas que faltan, con hora_actual_minutos y un origen opcional.
POST	/reoptimize-clima	Prueba candidatos de sustitución para las paradas afectadas por el clima y devuelve la ruta actual y la propuesta.

```
// POST /optimize
{ "modo_transporte": "pedestrian",
  "paradas": [ { "id": "parada_1", "lat": 14.6349, "lng": -90.5069,
                 "hora_apertura": "09:00", "hora_cierre": "18:00", "tiempo_estancia_minutos": 60 } ] }
// Respuesta 200
{ "orden_optimo": ["parada_1", "parada_3", "parada_2"], "distancia_total_km": 12.4 }
```

modo_transporte es auto (por defecto) o pedestrian. /reoptimize-clima admite de 1 a 25 paradas y hasta 5 grupos de sustitución con hasta 5 candidatos cada uno; si ningún candidato es viable, conserva la parada original e informa sin_sustitucion.

Estado	Causa
422	Validación (coordenadas fuera de rango, modo_transporte no admitido); no existe un orden que respete los horarios; una parada está fuera de la cobertura del mapa; o no hay ruta entre las paradas.
502	Valhalla devolvió una respuesta inválida (JSON, matriz incompleta o valores negativos).
503	Valhalla no responde, rechaza la conexión o supera VALHALLA_TIMEOUT_SEGUNDOS. No se recurre a cálculos aproximados.

4.12 Códigos de error del backend

HTTP	Código	Significado
400	PARAMETRO_INVALIDO	Parámetros o cuerpo inválidos; incluye detalles por campo. No se consulta ningún proveedor.
400	PROPUESTA_INVALIDA	Token de propuesta alterado o de otra instancia.
401	—	Token ausente, inválido o expirado (rutas protegidas).
404	RUTA_NO_ENCONTRADA	La ruta solicitada no existe.
409	PROPUESTA_OBSOLETA, REOPTIMIZACION_NO_DISPONIBLE	El itinerario cambió, o no hay ruta viable.
410	PROPUESTA_VENCIDA	La propuesta superó su vigencia.
422	DESTINO_NO_ENCONTRADO, SIN_LUGARES	Destino no hallado en Guatemala, o sin lugares disponibles.
500	ERROR_INTERNO	Error no controlado (sin detalles internos).
502	OPTIMIZADOR_NO_DISPONIBLE, RESPUESTA_OPTIMIZADOR_INVALIDA, PROVEEDOR_*	Falla del optimizador o de un proveedor externo.

HTTP	Código	Significado
503	SERVICIO_NO_CONFIGURADO, SERVICIO_RUTAS_NO_DISPONIBLE	Falta una clave de API, o Valhalla no está disponible.

5. Instalación y despliegue

5.1 Requisitos

Requisito	Detalle
Git	Para clonar el repositorio.
Docker Desktop	Levanta PostgreSQL, Redis, Valhalla y el optimizador. En Windows requiere virtualización habilitada y WSL 2.
Node.js 20 o superior	Para el backend y el frontend (se usan fetch, AbortController y node --test nativos). Con Node 22 se evitan advertencias en las pruebas del frontend.
Python 3.12	Solo si ejecuta el optimizador fuera de Docker (la imagen del contenedor usa python:3.12-slim).
Espacio y tiempo	La primera vez, Valhalla descarga el mapa de Guatemala y construye el grafo de rutas (unos 10 minutos). Su verificación de salud espera hasta 30 minutos al arrancar.
Claves de API	Ver sección 5.2. Solo Geoapify es obligatoria para iniciar el backend.

> **Si Docker no abre en Windows.** Revise en el Administrador de tareas (pestaña Rendimiento) que la virtualización esté activa; si no lo está, habilítela desde la BIOS/UEFI. Si el error continúa, ejecute wsl --status en PowerShell como administrador, instale WSL con wsl --install si hace falta y reinicie el equipo.

5.2 Claves de API

Servicio	Variable	Necesaria	Si falta
Geoapify	GEOAPIFY_API_KEY	Obligatoria	El backend no inicia y muestra la variable faltante.
OpenWeather	OPENWEATHER_API_KEY	Recomendada	El clima responde 503; priorizar-clima devuelve el orden original.
Google Gemini	GEMINI_API_KEY	Recomendada	Los itinerarios se arman con el planificador de respaldo; el asistente no está disponible.
SerpApi	SERPAPI_API_KEY	Opcional	Las recomendaciones de hoteles responden 503.

Servicio	Variable	Necesaria	Si falta
ExchangeRate-API	EXCHANGERATE_API_KEY	Opcional	Igual que SerpApi (se necesitan ambas para hoteles).
Foursquare	FOURSQUARE_API_KEY	Opcional	Solo se usan los lugares de OpenStreetMap (menos comercios y restaurantes).
Mapbox	VITE_MAPBOX_TOKEN (frontend)	Recomendada	El mapa no carga; la app muestra un aviso en lugar de fallar.

5.3 Puesta en marcha paso a paso

Los comandos están escritos para PowerShell (Windows). En macOS o Linux, reemplace Copy-Item por cp.

Paso 1. Clonar el repositorio y levantar los servicios de apoyo

```
git clone https://github.com/DerekMarmol/GuiaTuristicaProyecto.git
cd GuiaTuristicaProyecto

# Base de datos y caché
docker compose up -d postgres redis

# Motor de rutas (la primera vez descarga el mapa y construye el grafo: ~10 min)
docker compose up -d valhalla
Invoke-RestMethod http://localhost:8002/status

# Optimizador (construye la imagen y se conecta a Valhalla)
docker compose up -d --build optimizer
Invoke-RestMethod http://localhost:8000/health      # debe responder { status: "ok" }
```

Paso 2. Configurar y ejecutar el backend

```
cd backend
npm install
Copy-Item .env.example .env      # edite DATABASE_URL y las claves de API
npx prisma migrate dev           # aplica las migraciones y genera el cliente de Prisma
npm run dev                      # http://localhost:3000 (se reinicia al cambiar el código)
```

Cada vez que haga git pull y haya migraciones nuevas, vuelva a ejecutar npx prisma migrate dev.

Paso 3. Configurar y ejecutar el frontend

```
cd frontend
npm install
Copy-Item .env.example .env      # defina VITE_MAPBOX_TOKEN y VITE_MAPBOX_STYLE_URL
npm run dev                      # http://localhost:5173
```

Deje VITE_API_URL vacío: la aplicación llama a /api en su mismo origen y Vite reenvía esas peticiones al backend en http://localhost:3000.

Paso 4. Verificar

1. Abra http://localhost:5173 y complete los pasos 1 a 3 con un destino de Guatemala (por ejemplo, «Antigua Guatemala»).
2. En el paso 4 debe aparecer el itinerario generado. Si el aviso indica que se usó el «planificador automático», revise GEMINI_API_KEY.
3. Inicie sesión (o regístrese) para guardar el plan y verlo después en «Tus planes guardados».

5.4 Variables de entorno

Backend (backend/.env)

Variable	Por defecto	Descripción
GEOAPIFY_API_KEY	—	Clave de Geoapify. Obligatoria.
GEMINI_API_KEY, GEMINI_MODEL	—	Clave y modelo de Gemini. Sin clave se usa el planificador de respaldo.
OPENWEATHER_API_KEY	—	Clave de OpenWeather (pronóstico y clima actual).
SERPAPI_API_KEY, EXCHANGERATE_API_KEY	—	Claves para las recomendaciones de hoteles.
FOURSQUARE_API_KEY	—	Segunda fuente de lugares.
DATABASE_URL	—	Conexión de Prisma. Ejemplo local: postgresql://guia_admin:guiapassw@localhost:5432/guia_turistica?schema=public.
OPTIMIZER_URL	http://localhost:8000	URL base del optimizador.
PORT	3000	Puerto HTTP del backend.
HOST	::	Dirección de escucha (todas las interfaces). Use 127.0.0.1 para aceptar solo conexiones locales.
REDIS_URL	redis://localhost:6379	Conexión a Redis. No aparece en .env.example.

Variable	Por defecto	Descripción
JWT_SECRET, JWT_REFRESH_SECRET	valor de respaldo en el código	Firman los tokens de sesión. No aparecen en .env.example; defínalos siempre (ver sección 8.2).
JWT_EXPIRES_IN	2h	Duración del token de sesión.
CLIMA_REACTIVO_SECRETO	aleatorio por proceso	Firma de las propuestas de reordenamiento. Defínala si hay varias instancias.
CLIMA_REACTIVO_*	ver sección 2.5	Umbrales e intervalos del monitoreo (UMBRAL_PROBABILIDAD 0.7, INTERVALO_MINUTOS 12, ESPERA_PROPUESTAS_MINUTOS 30, VIGENCIA_PROPUESTA_MINUTOS 20, entre otros). Un valor fuera de rango impide arrancar.

Frontend (frontend/.env)

Variable	Por defecto	Descripción
VITE_MAPBOX_TOKEN	—	Token público de Mapbox para el mapa y las direcciones.
VITE_MAPBOX_STYLE_URL	—	Estilo del mapa.
VITE_API_URL	vacío	Solo si el backend está en otro dominio (por ejemplo, al publicar la app).
VITE_AUTH_SIMULADO	false	Con true, el registro y el inicio de sesión se simulan en el navegador (útil sin backend).

Optimizador (optimizer/.env)

Variable	Por defecto	Descripción
VALHALLA_URL	http://localhost:8002	URL de Valhalla. Con Docker Compose es http://valhalla:8002.
VALHALLA_TIMEOUT_SEGUNDOS	10	Tiempo máximo de espera de Valhalla.

5.5 Comandos útiles

Proyecto	Comando	Qué hace
Backend	npm run dev	Inicia el servidor con tsx y lo reinicia al detectar cambios.
Backend	npm run build y npm start	Compila a dist/ e inicia el servidor compilado (dist/backend/src/server.js).
Backend	npm test	Ejecuta las pruebas de test/ con las APIs externas simuladas.
Backend	npm run ai:test, npm run ai:tipos	Pruebas y comprobación estricta de tipos del módulo de IA.
Backend	npm run typecheck	Verifica los tipos de src/ y test/.
Backend	npx prisma migrate dev --name <nombre>	Crea y aplica una migración nueva.
Frontend	npm run dev	Servidor de desarrollo con recarga en caliente.
Frontend	npm run build	Comprueba tipos y genera la versión de producción con el service worker.
Frontend	npm test	Ejecuta las pruebas con node --test a través de tsx.
Optimizador	python -m pytest tests	Ejecuta las pruebas con pytest.
Docker	docker compose up -d / down	Levanta o detiene todos los servicios de apoyo.

5.6 Probar en el teléfono

El navegador solo entrega la ubicación (GPS) en páginas seguras (HTTPS), y la navegación guiada depende de ella. Para probar en un teléfono real, abra un túnel HTTPS hacia el servidor de Vite con Cloudflare Tunnel o ngrok. La configuración de Vite ya permite los dominios *.trycloudflare.com y *.ngrok-free.app. Una vez abierta en el teléfono, la aplicación puede instalarse como PWA desde el menú del navegador.

5.7 Compilación de producción

1. **Backend:** npm run build y luego npm start con las variables de entorno definidas en el servidor. Requiere acceso a PostgreSQL (ejecute npx prisma migrate deploy en el entorno de destino), a Redis y al optimizador.

2. **Frontend:** npm run build genera frontend/dist (archivos estáticos con el service worker). Debe servirse por HTTPS.
3. **Optimizador y Valhalla:** use docker compose up -d --build optimizer valhalla, o las mismas imágenes en el orquestador que se utilice.

> **Importante antes de publicar.** El repositorio está configurado para desarrollo. Antes de exponer el sistema: defina JWT_SECRET y JWT_REFRESH_SECRET, cambie las credenciales de PostgreSQL del docker-compose.yml, y decida cómo se comunicarán frontend y API (mismo origen, o VITE_API_URL más CORS en el backend). Consulte la sección 8.2.

5.8 Solución de problemas

Síntoma	Causa probable y solución
Docker Desktop no inicia	Virtualización desactivada o WSL ausente. Ver el recuadro de la sección 5.1.
503 SERVICIO_RUTAS_NO_DISPONIBLE al reoptimizar	Valhalla aún está construyendo el grafo (primer arranque) o no está levantado. Revise http://localhost:8002/status.
502 OPTIMIZADOR_NO_DISPONIBLE	El contenedor optimizer no está en marcha o OPTIMIZER_URL apunta a otra dirección.
503 SERVICIO_NO_CONFIGURADO	Falta una clave de API en backend/.env. Reinicie el backend después de editar el archivo.
El backend termina al iniciar	Falta GEOAPIFY_API_KEY; el mensaje indica la variable.
Error de Prisma por migración modificada	Se editó una migración ya aplicada. En desarrollo, npx prisma migrate reset (borra los datos locales).
El mapa no aparece	Falta VITE_MAPBOX_TOKEN; la app muestra un aviso en lugar del mapa.
La ubicación no se obtiene en el teléfono	La página no está en HTTPS. Use un túnel (sección 5.6) y acepte el permiso de ubicación.
Itinerario con el aviso «planificador automático»	Gemini sin clave, sin cuota o saturado. El plan es válido, pero sin IA.
La prueba «espacia el inicio de las consultas» falla a veces	Es una prueba inestable conocida, sobre todo en Windows (depende de la precisión del reloj). Basta con volver a ejecutar npm test; no indica un error del backend.

6. Frontend

6.1 Tecnologías

Tecnología	Uso
React 18 + TypeScript	Interfaz con componentes funcionales y hooks. No usa router ni librería de estado global.
Vite 5	Servidor de desarrollo, empaquetado y proxy hacia /api.
Mapbox GL 3	Mapa vectorial con capas propias (ruta del día, tramo de navegación, marcadores).
vite-plugin-pwa	Service worker, precarga de archivos y actualización automática.
Fuentes	Manrope y Barlow Semi Condensed (paquetes @fontsource, sin depender de una CDN).
node:test + tsx	Pruebas unitarias de la lógica de servicios y utilidades.

6.2 Estructura de frontend/src

Carpeta	Contenido
pages/	Una pantalla por paso (Paso1Destino, Paso2Hospedaje, Paso3Contexto, Paso4Generacion) y PantallaAuth.
components/	Piezas de interfaz reutilizables: mapa, paneles, chat, navegación, clima, recomendaciones.
hooks/	Lógica con efectos: GPS, monitoreo del clima, navegación, punto de partida, lugares abiertos.
services/	Llamadas a la API y lógica de dominio (conversión de respuestas, cálculos, estado de controladores).
context/	ViajeContext (el viaje y el itinerario) y AuthContext (la sesión).
utils/	Fechas, horarios, almacenamiento local, detección de transporte y estimación de velocidad GPS.
types/	Tipos propios del frontend (auth, chat, gasto, calificación, recomendación).

6.3 Flujo de pantallas

La aplicación no usa rutas de URL: el paso actual (1 a 4) es un valor del ViajeContext y App.tsx muestra la pantalla correspondiente. PantallaAuth se superpone cuando el usuario pulsa «Iniciar sesión». Cada pantalla muestra un Stepper con el avance.

Paso	Pantalla	Qué captura o muestra
1. Destino	Paso1Destino	Destino, fecha de inicio y fecha de fin (validación: no pasadas y fin igual o posterior al inicio). Con sesión iniciada ofrece «Tus planes guardados» (MisPlanes).
2. Hospedaje	Paso2Hospedaje	Tres opciones: ya tengo hospedaje (pide el nombre del hotel), necesito hotel, o no necesito hotel.
3. Contexto	Paso3Contexto	Tipo de viaje (solo, pareja, familia, amigos), presupuesto, medio de transporte (5 opciones), intereses (10 opciones) y módulos opcionales: reordenar por clima, «Abierto ahora», asistente en ruta, panel de presupuesto y modo Memoria.
4. Itinerario	Paso4Generacion	Genera el plan (con indicador de progreso), permite guardarlo y exportarlo, y se organiza en cuatro secciones con barra de navegación inferior.

Las cuatro secciones del paso 4 son:

Sección	Contenido
Itinerario	Plan por días con horarios de llegada y salida, alertas de horario ajustado o descartado, indicador del clima del día, calificación de paradas y traslado desde el punto de partida.
Mapa	Mapa con las paradas, la posición en vivo, la ruta de navegación, el punto de partida elegible y el botón «Abierto ahora».
Dormir y comer (o «Comer»)	Recomendaciones de hoteles (solo si el usuario necesita hotel) y de restaurantes por parada.
En ruta	Propuesta de cambio por clima, recálculo de la ETA, telemetría de velocidad y panel de presupuesto con gastos.

6.4 Estado global y persistencia

Elemento	Descripción
ViajeContext	Guarda el borrador del viaje (viaje), el paso actual, los itinerarios generados, las calificaciones y los gastos. Expone funciones como actualizarViaje, irAPaso, setItinerarios, cargarPlan y reiniciar. Al reemplazar el itinerario por uno nuevo se reinician calificaciones y gastos.
AuthContext	Guarda el usuario y el token de la sesión en localStorage (clave guia.sesion). Expone iniciarSesion, registrar y cerrarSesion.
Persistencia del viaje	El viaje en curso se guarda en el teléfono (clave guia.viaje, con número de versión) para sobrevivir a recargar la página. Si cambia la forma de lo guardado, se sube la versión y se descartan los datos viejos.
Estado del paso 4	La sección abierta, el día visible y las paradas visitadas se guardan con el prefijo guia.paso4..
Tolerancia a fallos	Si el almacenamiento no está disponible (navegación privada o espacio lleno), la app sigue funcionando sin recordar.

6.5 Servicios

Archivo	Responsabilidad	Destino
auth.ts	Registro e inicio de sesión (con modo simulado)	/api/auth/registro, /login
itinerario.ts	Generar, convertir respuesta, guardar y exportar planes	/api/itinerarios/generar, /guardar, /:id/exportar
planes.ts	Listar y abrir planes guardados	/api/itinerarios/mis-planes, /:id
clima.ts	Pronóstico y clima actual; umbral de 10 mm para avisar de lluvia	/api/clima/pronostico, /actual
climaReactivo.ts, climaReactivoEstado.ts	Evaluar, proponer y aceptar cambios por clima; construcción del itinerario activo	/api/itinerarios/clima-reactivo/*
recomendaciones.ts	Hoteles y restaurantes	/api/hoteles/…, /api/restaurantes/…

Archivo	Responsabilidad	Destino
reoptimizacion.ts	Recalcular la ruta ante clima, retraso o parada cancelada	/api/itinerarios/reoptimizar
eta.ts	Recalcular la hora estimada de llegada	/api/itinerarios/recalcular-eta
puntoPartida.ts	Traslado desde el punto de partida (distancia máxima de 50 km)	/api/itinerarios/traslado
abiertosAhora.ts, abiertosAhoraEstado.ts	Consulta y controlador de estado de «Abierto ahora»	/api/lugares/abiertos-ahora
navegacion.ts	Ruta por calles, indicaciones en español, progreso y detección de desvío	Mapbox Directions (directo)
ubicacionActual.ts	Posición del navegador (espera máxima de 15 s; antigüedad máxima de 5 min)	API de geolocalización
chat.ts	Respuestas simuladas del asistente (modo demostración)	— (pendiente de conectar con /api/chat)

6.6 Hooks

Hook	Función
useMonitoreoClima	Revisa el clima al abrir el paso 4 y luego cada intervalo que indica el backend, solo si el día mostrado es hoy y quedan paradas pendientes. Evita revisiones simultáneas y descarta respuestas tardías. Solo funciona con la app abierta.
useNavegacion	Pide la ruta a Mapbox, avanza las indicaciones según el GPS, recalcula si el usuario se desvía y detecta la llegada.
useUbicacionEnVivo	Sigue la posición del usuario mientras la sección Mapa está activa.
usePuntoPartida	Resuelve el punto de partida (ubicación actual o punto elegido en el mapa) y su traslado a la primera parada.
useAbiertosAhora	Controla el modo «Abierto ahora»; la ubicación se pide solo al activarlo o al pulsar «Actualizar ubicación».
useTelemetria	Estima la velocidad a partir del GPS (usa coords.speed o la calcula con distancia y tiempo entre lecturas).

6.7 Componentes

Componente	Función
Mapa, MapaSeguro	Mapa de Mapbox con paradas, ruta y posición; MapaSeguro muestra un aviso si falta el token en lugar de romper la app.
Stepper	Indicador del paso actual (1 a 4).
NavegacionPaso4	Barra inferior para cambiar de sección en el paso 4.
BarraNavegacion	Indicación actual de la navegación guiada sobre el mapa (maniobra, distancia, duración).
BarraPuntoPartida, TrasladoDesdePartida	Elección del punto de partida y desglose de «Traslado y margen de seguridad».
IndicadorClima	Pronóstico del día y clima «Ahora en tu destino».
PropuestaClima	Muestra el nivel de riesgo, las paradas afectadas, los cambios propuestos y los botones «Aceptar cambios» y «Mantener mi itinerario».
AbiertosAhora	Botón y lista de lugares abiertos en este momento.
AlertaHorario	Aviso de visita con horario ajustado o descartada.
Recomendaciones	Hoteles y restaurantes recomendados.
PanelEta	Recálculo de la hora estimada de llegada.
PanelTelemetria	Velocidad y estado del GPS.
PanelPresupuesto	Registro de gastos y comparación con lo planeado.
CalificacionParada	Calificación de cada parada visitada.
MisPlanes	Planes guardados de la cuenta.
ChatFlotante	Chat flotante con el asistente de viaje y sugerencias rápidas.
OfflineBanner	Aviso cuando el dispositivo pierde la conexión.
FondoMapa	Fondo ilustrado de la aplicación (SVG).

6.8 Aplicación web progresiva (PWA)

Aspecto	Configuración
Manifiesto	public/manifest.json: nombre «Guía Turística Inteligente», display: standalone, orientación vertical, idioma es, colores de tema y fondo #fef1da.

Aspecto	Configuración
Íconos	192 px, 512 px y 512 px *maskable* (public/icons/), más favicon.svg y el ícono para iOS.
Service worker	vite-plugin-pwa con registerType: "autoUpdate"; la app lo registra al iniciar (registerSW({ immediate: true })).
Precarga	Se guardan JS, CSS, HTML, SVG, PNG, JSON y fuentes; navigateFallback apunta a /index.html y se limpian cachés antiguas.
Límite de archivo	Se eleva a 4 MiB porque Mapbox GL hace que el paquete principal supere los 2 MiB por defecto.
Sin conexión	OfflineBanner avisa de la falta de conexión; la generación de itinerarios y las consultas requieren red.

6.9 Pruebas del frontend

El frontend tiene 9 archivos de prueba en frontend/test/: lugares abiertos ahora, navegación, clima, clima reactivo, recomendaciones, planes guardados, punto de partida, itinerario y velocidad por GPS. Prueban la lógica de servicios y utilidades (no renderizan componentes) y no consumen cuotas de APIs.

7. Pruebas y calidad

Proyecto	Herramienta y archivos	Qué cubren
Backend	node:test con tsx; 35 archivos en test/ y 3 en src/ai/	Lugares, horarios de apertura, clasificación y priorización climática, clima reactivo, generación de itinerarios, fallos de IA, hoteles, restaurantes, caché con Redis, autenticación, usuarios, planes guardados, exportación, chat, reoptimización, ETA y traslado.
Frontend	node:test con tsx; 9 archivos	Lógica de servicios y utilidades (ver sección 6.9).
Optimizador	pytest; 9 archivos en tests/	Optimización, reoptimización, clima, matriz de Valhalla, errores del servicio de rutas y pruebas de integración con un Valhalla simulado (valhalla_falso.py).

Todas las pruebas simulan Geoapify, OpenWeather, SerpApi, ExchangeRate-API, Gemini, el optimizador y Valhalla, de modo que no requieren claves ni consumen cuota. Para ejecutarlas, vea la sección 5.5.

Además, npm run typecheck (backend) y npm run build (frontend, que ejecuta tsc --noEmit) comprueban los tipos de todo el código, incluidos los contratos de shared/types.

8. Limitaciones y recomendaciones

8.1 Limitaciones conocidas del producto

- **Alcance geográfico:** solo Guatemala (geocodificación, mapa de Valhalla y moneda).
- **Modos de transporte:** el optimizador calcula con dos modos, auto y a pie. Moto, transporte público y transporte aéreo se calculan como auto.
- **Horarios:** dependen de lo que publican Geoapify y OpenStreetMap; en Guatemala muchos lugares no tienen horario. Se les asigna uno típico y se marca horarioEstimado; las alternativas por clima exigen horario confirmado, por lo que en zonas con pocos datos habrá pocas propuestas.
- **Precios de hoteles y restaurantes:** son referenciales y pueden diferir del cargo final.
- **Clima reactivo:** funciona solo con la aplicación abierta y solo el día del itinerario; usa una ubicación de clima por revisión. La propuesta reduce la exposición a la lluvia, pero no garantiza la seguridad ante tormentas.
- **Asistente en el frontend:** ChatFlotante usa respuestas de demostración (services/chat.ts); el endpoint real POST /api/chat está implementado en el backend pero aún no está conectado.
- **Calificaciones y gastos:** se guardan solo en el teléfono; el código marca como pendiente guardarlos en el servidor.
- **Compartir planes:** la base de datos tiene los campos esPublico y codigoCompartido, pero no existe un endpoint que los use.
- **Endpoints sin consumidor:** POST /api/itinerarios/validar y /priorizar-clima están implementados y probados, pero el frontend actual no los llama.
- **Estado en_curso de las paradas:** el frontend no lo persiste; el paso 4 envía las paradas como completadas o pendientes.

8.2 Observaciones técnicas y recomendaciones

Resultado del análisis del código. No son errores que impidan usar el sistema en desarrollo, pero conviene atenderlos antes de publicarlo.

Observación	Recomendación
Secretos JWT con valor de respaldo en el código. Si JWT_SECRET o JWT_REFRESH_SECRET no están definidos, se usan textos fijos escritos en auth.plugin.ts y auth.routes.ts, que son públicos en el repositorio.	Definirlos siempre en el entorno con valores largos y aleatorios, y añadirlos a .env.example. Valorar que el servidor no arranque en producción si faltan.
Variables no documentadas. REDIS_URL, JWT_SECRET, JWT_REFRESH_SECRET y JWT_EXPIRES_IN se leen en el código pero no figuran en .env.example.	Agregarlas al archivo de ejemplo con comentarios.
Credenciales de desarrollo en docker-compose.yml (usuario y contraseña de PostgreSQL).	Cambiarlas en cualquier entorno compartido o público y moverlas a variables de entorno.
CORS. @fastify/cors está instalado pero no se registra en app.ts; en desarrollo no hace falta porque Vite reenvía /api.	Si el frontend se publica en otro dominio que la API, registrar CORS con una lista de orígenes permitidos, o servir ambos bajo el mismo origen.
Renovación de sesión. El token dura 2 horas y el frontend no usa /api/auth/renovar (guarda solo token y usuario); al vencer, las rutas protegidas fallan hasta volver a iniciar sesión.	Guardar el refreshToken y renovar el token de forma automática.

Observación	Recomendación
Validación de cuentas. El registro solo comprueba que los campos no estén vacíos; no valida el formato del correo ni la fortaleza de la contraseña, y no se observa limitación de intentos de inicio de sesión.	Validar con Zod (como el resto de la API) y limitar los intentos por IP o cuenta.
Formato de errores desigual. Las rutas de autenticación, usuarios y guardado responden { statusCode, mensaje }; el resto, { error: { codigo, mensaje } }.	Unificar con ErrorApi para que el frontend maneje un solo formato.
Varias instancias de PrismaClient. Se crea una en auth.routes.ts, otra en usuarios.routes.ts y otra en itinerarios.ts.	Compartir una sola instancia (por ejemplo, un módulo lib/prisma.ts) para no multiplicar conexiones.
Prueba inestable espacia el inicio de las consultas (test/horarios.test.ts).	Dar más margen a la prueba o comprobar que cada consulta empiece en su hora asignada o después, en lugar de medir la separación entre dos inicios seguidos.

8.3 Trabajo futuro sugerido

- Conectar ChatFlotante con POST /api/chat.
- Implementar el compartir planes con código (esPublico, codigoCompartido).
- Guardar calificaciones y gastos en el servidor.
- Notificaciones en segundo plano para el clima reactivo (hoy requiere la app abierta).
- Usar PostGIS para las consultas de cercanía si crece el volumen de datos propios.
- Distinguir más modos de transporte (moto, transporte público) en el cálculo de rutas.

Anexo. Documentos relacionados del repositorio

Documento	Contenido
README.md	Requisitos, puesta en marcha y resumen de la API.
backend/README.md	Referencia detallada de cada endpoint: parámetros, reglas de validación, ejemplos en PowerShell, límites de consumo y limitaciones (más de 2000 líneas).
optimizer/API.md	Contrato de /optimize, /reoptimize y /reoptimize-clima, y sus errores.
docs/valhalla.md	Configuración de Valhalla, preparación de datos, consultas de ejemplo, pruebas y errores habituales.
docs/informe-unificacion-clima.md	Informe sobre la unificación de la clasificación del clima.
docs/Guia-Turistica-Documentacion.pdf	Resumen del proyecto y justificación de cada tecnología elegida (8 páginas).
database/erd.md	Diagrama entidad-relación original en Mermaid.